

JSON Module

JSON is a syntax for storing and exchanging data. JSON is text, written with JavaScript object notation. Python has a built-in package called json, which can be used to work with JSON data.

Parse JSON - Convert from JSON to Python

If you have a JSON string, you can parse it by using the json.loads() method.

```
import json
# some JSON:
x = '{ "name":"John", "age":30, "city":"New York"}'
# parse x:
y = json.loads(x)
print(y)
# the result is a Python dictionary:
print(y["age"])
```

output:

```
{'name': 'John', 'age': 30, 'city': 'New York'}
30
```

Convert from Python to JSON

If you have a Python object, you can convert it into a JSON string by using the json.dumps() method.

Convert from Python to JSON:

```
import json
# a Python object (dict):
x = {
```

```
"name": "John",
"age": 30,
"city": "New York"
}

# convert into JSON:
y = json.dumps(x)
# the result is a JSON string:
print(y)
```

output:

```
{"name": "John", "age": 30, "city": "New York"}
```

You can convert Python objects of the following types, into JSON strings:

dict
list
tuple
string
int
float
True/False
None

Convert Python objects into JSON strings, and print the values

```
import json
print(json.dumps({"name": "John", "age": 30}))
print(json.dumps(["apple", "bananas"]))
print(json.dumps(("apple", "bananas")))
print(json.dumps("hello"))
print(json.dumps(42))
print(json.dumps(31.76))
```

```
print(json.dumps(True))
print(json.dumps(False))
```

output:

```
{"name": "John", "age": 30}
```

```
["apple", "bananas"]
```

```
["apple", "bananas"]
```

```
"hello"
```

```
42
```

```
31.76
```

```
true
```

```
false
```

Convert a Python object containing all the legal data types

```
import json
x = {
    "name": "John",
    "age": 30,
    "married": True,
    "divorced": False,
    "children": ("Ann", "Billy"),
    "pets": None,
    "cars": [
        {"model": "BMW 230", "mpg": 27.5},
        {"model": "Ford Edge", "mpg": 24.1}
    ]
}

print(json.dumps(x))
```

output:

```
{"name": "John", "age": 30, "married": true, "divorced": false, "children": ["Ann", "Billy"], "pets": null, "cars": [{"model": "BMW 230", "mpg": 27.5}, {"model": "Ford Edge", "mpg": 24.1}]}
```

Format the Result

The example above prints a JSON string, but it is not very easy to read, with no indentations and line breaks. The `json.dumps()` method has parameters to make it easier to read the result. Use the `indent` parameter to define the numbers of indents.

```
import json
x = {
    "name": "John",
    "age": 30,
    "married": True,
    "divorced": False,
    "children": ("Ann","Billy"),
    "pets": None,
    "cars": [
        {"model": "BMW 230", "mpg": 27.5},
        {"model": "Ford Edge", "mpg": 24.1}
    ]
}

# use four indents to make it easier to read the result:
print(json.dumps(x, indent=4))
```

output:

```
{
    "name": "John",
    "age": 30,
    "married": true,
```



```

    "divorced": false,
    "children": [
        "Ann",
        "Billy"
    ],
    "pets": null,
    "cars": [
        {
            "model": "BMW 230",
            "mpg": 27.5
        },
        {
            "model": "Ford Edge",
            "mpg": 24.1
        }
    ]
}

```

You can also define the separators, default value is (" , " , ":"), which means using a comma and a space to separate each object, and a colon and a space to separate keys from values. Use the separators parameter to change the default separator:

```

import json
x = {
    "name": "John",
    "age": 30,
    "married": True,

```

```

"divorced": False,
"children": ("Ann","Billy"),
"pets": None,
"cars": [
    {"model": "BMW 230", "mpg": 27.5},
    {"model": "Ford Edge", "mpg": 24.1}
]
}

```

use . and a space to separate objects, and a space, a = and a space to separate keys from their values:

```
print(json.dumps(x, indent=4, separators=(". ", " = ")))
```

output:

```

{
    "name" = "John".
    "age" = 30.
    "married" = true.
    "divorced" = false.
    "children" = [
        "Ann".
        "Billy"
    ].
    "pets" = null.
    "cars" = [
        {
            "model" = "BMW 230".
            "mpg" = 27.5
        }.
    ]
}

```

```

        "model" = "Ford Edge".
        "mpg" = 24.1
    }
]
}

```

Order the Result

The `json.dumps()` method has parameters to order the keys in the result. Use the `sort_keys` parameter to specify if the result should be sorted or not.

```

import json
x = {
    "name": "John",
    "age": 30,
    "married": True,
    "divorced": False,
    "children": ("Ann","Billy"),
    "pets": None,
    "cars": [
        {"model": "BMW 230", "mpg": 27.5},
        {"model": "Ford Edge", "mpg": 24.1}
    ]
}

# sort the result alphabetically by keys:
print(json.dumps(x, indent=4, sort_keys=True))

```

output:

```

{
    "age": 30,
    "cars": [

```

```
{
  "model": "BMW 230",
  "mpg": 27.5
},
{
  "model": "Ford Edge",
  "mpg": 24.1
}
],
"children": [
  "Ann",
  "Billy"
],
"divorced": false,
"married": true,
"name": "John",
"pets": null
}
```

Regular Expression

A RegEx, or Regular Expression, is a sequence of characters that forms a search pattern. RegEx can be used to check if a string contains the specified search pattern.

RegEx Module

Python has a built-in package called re, which can be used to work with Regular Expressions. When you have imported the re module, you can start using regular expressions.

eg: Search the string to see if it starts with "The" and ends with "Spain"

RegEx Functions

The re module offers a set of functions that allows us to search a string for a match:

<code>re.search()</code>	<i>Searches for a match anywhere in the string</i>
<code>re.match()</code>	<i>Matches a pattern only at the beginning</i>
<code>re.findall()</code>	<i>Returns all non-overlapping matches</i>
<code>re.sub()</code>	<i>Replaces matches with a string</i>
<code>re.split()</code>	<i>Splits a string by the pattern</i>
<code>re.compile()</code>	<i>Compiles a pattern for reuse</i>

re.findall()

Returns a list containing all matches

eg: Print a list of all matches:

```
import re
txt = "The rain in Spain"
x = re.findall("ai", txt)
print(x)
```

output:

```
['ai', 'ai']
```

eg: Print a list of all digit matches:

```
text = "123 Main Street, Apt 4B, 456 Oak Ave"
numbers = re.findall(r'\d+', text)
print("Numbers:", numbers) # Output: Numbers: ['123', '4', '456']
```

re.search()

Returns a Match object if there is a match anywhere in the string

```
import re
txt = "The rain in Spain"
x = re.search("\s", txt)
print("The first white-space character is located in position:", x.start())
```

output:

The first white-space character is located in position: 3

re.split()

Returns a list where the string has been split at each match.

Split at each white-space character:

```
import re
txt = "The rain in Spain"
x = re.split("\s", txt)
print(x)
```

output:

['The', 'rain', 'in', 'Spain']

re.sub()

Replaces one or many matches with a string

Replace every white-space character with the number ','

```
import re
txt = "The rain in Spain"
x = re.sub("\s", ",", txt)
print(x)
```

output:

The,rain,in,Spain

#Search for a sequence that starts with "he", followed by two (any) characters, and an "o":

```
import re
txt = "hello planet"
x = re.findall("he..o", txt)
print(x)
```

output:

['hello']

#Search for a sequence that starts with "he", followed by two (any) characters, and an "o"

```
import re
txt = "hello planet"
x = re.findall("he..o", txt)
print(x)
```

output:


```
['hello']
```

^ - Starts with

#Search for a sequence that starts with "hello"

```
import re
txt = "hello planet"
x = re.findall("^hello", txt)
if x:
    print("Yes, the string starts with 'hello'")
else:
    print("No match")
```

output:

Yes, the string starts with 'hello'

#Search for a sequence that starts with "he", followed by 0 or more (any) characters, and an "o"

```
import re
txt = "heo helo hello"
x = re.findall("he.*o", txt)
print(x)
```

output:

```
['heo helo hello']
```

+ - One or more occurrences

#Search for a sequence that starts with "he" followed by 1 or more (any) characters, and an "o"

```
import re
txt = "ho heo helo hello"
```

```
x = re.findall("he.+o", txt)
print(x)
```

output:

```
['heo helo hello']
```

?-Zero or one occurrences

#Search for a sequence that starts with "he", followed by 0 or 1 (any) character, and an "o"

```
import re
txt = "heo helo hello"
x = re.findall("he.?o", txt)
print(x)
```

output:

```
['heo', 'heLo']
```

{}- Exactly the specified number of occurrences

#Search for a sequence that starts with "he", followed exactly 2 (any) characters, and an "o"

```
import re
txt = "helo hello"
x = re.findall("he.{2}o", txt)
print(x)
```

output:

```
['hello']
```

\$- Ends with

#search for a sequence that ends with "planet"

```
import re
txt = "hello planet"
#Check if the string ends with 'planet':
x = re.findall("planet$", txt)
if x:
    print("Yes, the string ends with 'planet'")
else:
    print("No match")
```

output:

Yes, the string ends with 'planet'

|-Either or

#Check if the string contains either "falls" or "stays"

```
import re
txt = "The rain in Spain falls mainly in the plain!"
x = re.findall("falls|stays", txt)
print(x)
if x:
    print("Yes, there is at least one match!")
else:
    print("No match")
```

output:

['falls']

Yes, there is at least one match!

Sets

A set is a set of characters inside a pair of square brackets [] with a special meaning.

[arn] Returns a match where one of the specified characters (a, r, or n) is present

```
import re
txt = "The rain in Spain"
#Check if the string has any a, r, or n characters:
x = re.findall("[arn]", txt)
print(x)

if x:
    print("Yes, there is at least one match!")
else:
    print("No match")
```

output:

```
['r', 'a', 'n', 'n', 'a', 'n']
Yes, there is at least one match!
```

[a-n] Returns a match for any lower case character, alphabetically between a and n

```
import re
txt = "The rain in Spain"
#Check if the string has any characters between a and n:
x = re.findall("[a-n]", txt)
print(x)
if x:
    print("Yes, there is at least one match!")
else:
```

```
print("No match")
```

output:

```
['h', 'e', 'a', 'i', 'n', 'i', 'n', 'a', 'i', 'n']
```

Yes, there is at least one match!

[^arn] Returns a match for any character EXCEPT a, r, and n

```
import re
txt = "The rain in Spain"
#Check if the string has other characters than a, r, or n:
x = re.findall("[^arn]", txt)
print(x)
if x:
    print("Yes, there is at least one match!")
else:
    print("No match")
```

output:

```
['T', 'h', 'e', ' ', 'i', ' ', 'i', ' ', 'S', 'p', 'i']
```

Yes, there is at least one match!

[0123] Returns a match where any of the specified digits (0, 1, 2, or 3) are present

```
import re
txt = "The rain in Spain"
#Check if the string has any 0, 1, 2, or 3 digits:
x = re.findall("[0123]", txt)
print(x)
if x:
    print("Yes, there is at least one match!")
```

```
else:
    print("No match")
```

output:

[]

No match

[0-9] Returns a match for any digit between 0 and 9

```
import re
txt = "8 times before 11:45 AM"
#Check if the string has any digits:
x = re.findall("[0-9]", txt)
print(x)
if x:
    print("Yes, there is at least one match!")
else:
    print("No match")
```

output:

['8', '1', '1', '4', '5']

Yes, there is at least one match!

[0-5][0-9] Returns a match for any two-digit numbers from 00 and 59

```
import re
txt = "8 times before 11:45 AM"
#Check if the string has any two-digit numbers, from 00 to
59:
x = re.findall("[0-5][0-9]", txt)
```

```

print(x)
if x:
    print("Yes, there is at least one match!")
else:
    print("No match")

```

output:

```
['11', '45']
```

Yes, there is at least one match!

[a-zA-Z] Returns a match for any character alphabetically between a and z, lower case OR upper case

```

import re
txt = "8 times before 11:45 AM"
#Check if the string has any characters from a to z lower
case, and A to Z upper case:
x = re.findall("[a-zA-Z]", txt)
print(x)
if x:
    print("Yes, there is at least one match!")
else:
    print("No match")

```

output:

```
['t', 'i', 'm', 'e', 's', 'b', 'e', 'f', 'o', 'r', 'e', 'A', 'M']
```

Yes, there is at least one match!